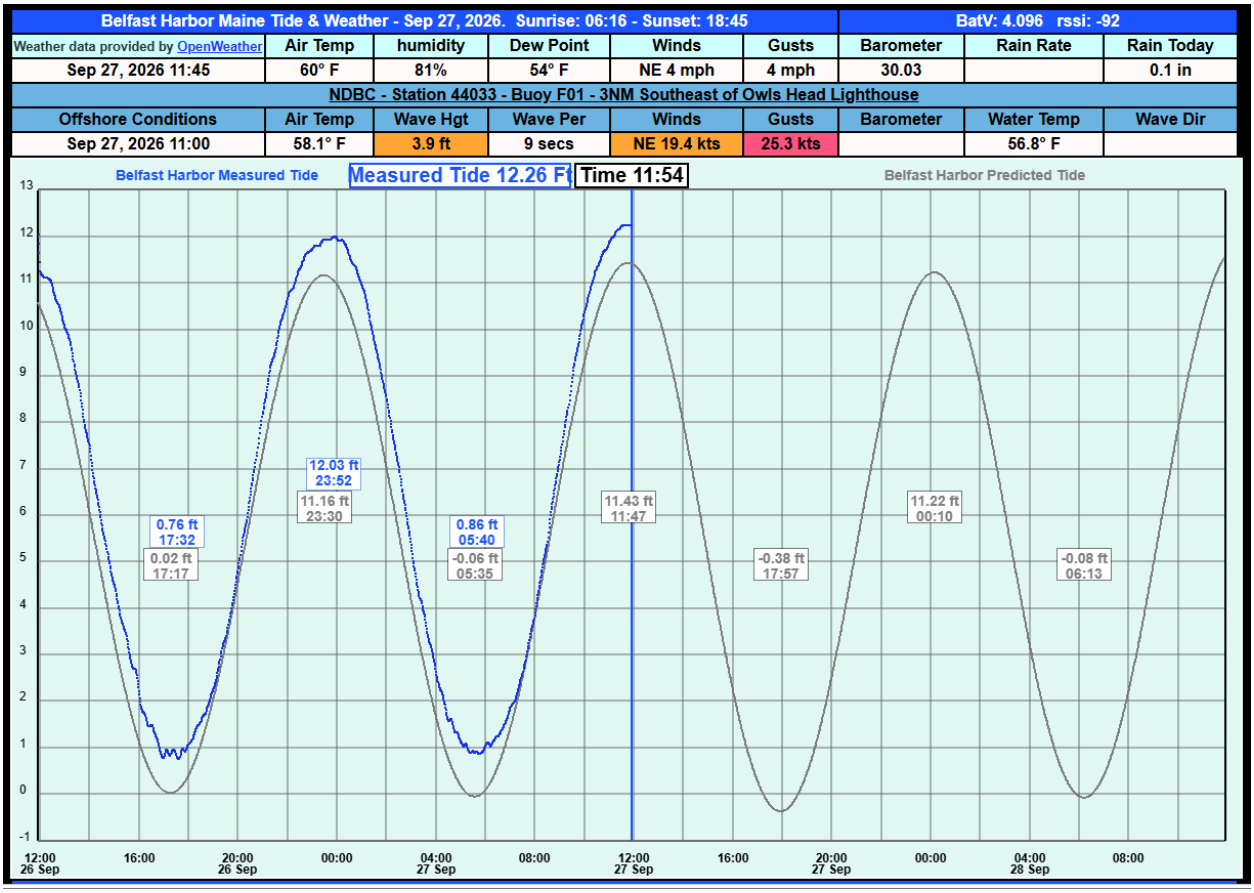
ACTION

Atlantic Coastal Tide Instrumentation and Observation Network

Technical Reference Manual

Software, Firmware, and Operations Reference



Example station web page (tide.html), generated by tide.py: Belfast Harbor, September 27, 2026

Revision date: November 1, 2026
Source repository: github.com/cfi1513840/tide-gauge

Table of Contents

Table of Contents 2
1. Introduction 5
1.1 Purpose of This Manual 5
1.2 Audience 5
1.3 Companion Document 5
1.4 System Summary 5
2. IT Technician Operations Guide 6
2.1 Is the System Actually Running? 6
2.2 Is the Website Actually Updating? 6
2.3 Common Remedial Actions 7
Restarting the main service 7
Rebooting the Raspberry Pi 7
Checking recent log output 7
2.4 Remote Access 7
2.5 What NOT To Do Without Escalating 8
2.6 Escalation 8
3. System Overview 9
3.1 High-Level Description 9
3.2 Sensor Data Paths 9
Path A: LoRa radio 9
Path B: Blues Notecard through the Pi 9
Path C: Blues Notecard directly to InfluxDB Cloud 10
Path D: InfluxDB Cloud back to the Pi 10
3.3 From Ingestion to Display 10
3.4 Main Loop Schedule 10
4. Hardware and Firmware 12
4.1 Physical Components 12
4.2 Firmware Sketches 12
5. Software Architecture 13
5.1 Core Modules 13
5.2 CGI Scripts (Web-Facing) 14
5.3 Installation and Maintenance Utilities 14
6. Data Storage 15
6.1 SQLite 15
6.2 InfluxDB 15

Local: InfluxDB 3 Core ... 16
Reading fields ... 16
Cloud: InfluxDB Cloud Serverless ... 16
Viewing the data ... 17
6.3 Outlier Rejection ... 17
Phantom readings ... 18
7. Security and Encryption ... 19
7.1 Overview ... 19
7.2 Key Management ... 19
7.3 Password Hashing and Legacy Migration ... 19
7.4 Mail Spool ... 20
7.5 Station Secrets and Network Exposure ... 20
8. Web Presentation Layer ... 21
8.1 Pages ... 21
8.2 CGI Scripts ... 21
8.3 Python 3.13 Compatibility ... 21
8.4 Cloudflare Tunnel ... 21
8.5 Site-Specific Customization ... 22
webimage.png ... 22
webinfo.txt (optional) ... 22
STATION_LOCATION (tide.env) ... 22
9. Alert System ... 24
9.1 Email Delivery ... 24
9.2 SMS Delivery ... 24
9.3 Alert Evaluation ... 24
10. External Services and Accounts ... 26
11. Deployment and Installation ... 27
11.1 install.sh ... 27
11.2 Python Package Requirements ... 27
11.3 Upgrading a Station ... 28
11.4 Source Branches ... 28
12. Known Technical Debt ... 29
cgi.FieldStorage() and the Python 3.13 removal ... 29
SQL built from strings in processalerts.cgi ... 29
Hardcoded time zone ... 29
Failed alerts are not retried ... 29
Automatic failover needs updating for three sensors ... 29

Duplicate sun-calculation packages 29
Inactive code retained deliberately 29
ndbcdata is not trimmed 29
tidemonitor.py and sensor types 29
Outlier baseline can lock onto a steady false reading 29
13. Appendices 31
Appendix A -- tide.env Reference 31
Appendix B -- Services and Scheduled Jobs 32
Appendix C -- Files and Locations 32
Appendix D -- Source Repository 32

1. Introduction

1.1 Purpose of This Manual

This manual documents the software, firmware, and operational architecture of ACTION (Atlantic Coastal Tide Instrumentation and Observation Network) tide and weather monitoring stations. It is a working reference for anyone responsible for keeping a deployed station running. That is mainly an on-site IT technician who did not build the system and may not have routine access to the development team. It is also useful to anyone extending or modifying the software.

The material most likely to be needed during an actual incident comes first, in Section 2: checking whether the system is healthy, and what to do if it isn't. Everything after that builds up the architectural and module-level detail needed for deeper troubleshooting or future development.

1.2 Audience

This manual assumes the reader is technically capable (comfortable with a command line, SSH, and basic Linux system administration) but has no prior familiarity with this specific codebase. It does not assume Python development experience, though some is helpful for anything beyond the operational tasks in Section 2.

The most common real-world scenario this manual is written for: a station stops updating or otherwise misbehaves, for example one operated autonomously by a school or institution without a developer on site. Someone with general IT skills but no history with the project needs to work out what's wrong and, ideally, fix it without reaching the original developer.

1.3 Companion Document

Building a new station, or upgrading an existing one to a new code branch, is covered separately in the ACTION Installation Manual (`ACTION_Installation_Manual.md`, in the repository root). It also covers the external account setup, the step-by-step prerequisite tutorials, and a running list of installation-specific issues and fixes. This manual explains how a running station works and how to keep it running; it does not repeat installation procedures.

1.4 System Summary

An ACTION station consists of one or more ultrasonic sensors that report water-level distance readings over either a LoRa radio link or a Blues Notecard cellular connection. A Raspberry Pi receives, processes, and stores those readings. It also serves a web dashboard (tide plots, weather, and an optional email/SMS alert subscription system) and, where a screen is attached, a local Tk display.

A single long-running Python process, tide.py, is the heart of the system. It ingests sensor data, writes it to a local SQLite database and a local InfluxDB 3 Core time-series database, generates the web content, evaluates alerts, and periodically syncs new readings to a shared InfluxDB Cloud database. All of this runs from one continuously running process managed by systemd. A headless alternative, tidemonitor.py, can run instead of tide.py where only data collection is needed (Section 5.1).

> **NOTE:** Sensor data reaches the station in three ways: a LoRa radio link (sensor to a local receiver attached to the Pi by USB), a Blues Notecard cellular link (sensor to Blues Notehub.io, which forwards each batch of readings to tide.py over HTTPS), or by reading a sensor's data back from InfluxDB Cloud. A Notecard sensor can also be routed from Notehub directly to InfluxDB Cloud, bypassing the Pi entirely. See Section 3.2.

2. IT Technician Operations Guide

This section is deliberately placed before the architectural detail. If the station appears to be down or misbehaving, start here.

2.1 Is the System Actually Running?

The main process runs as a systemd service named tide. It starts automatically at boot, and the current service template (`tide.service.template`) also restarts it automatically if the process crashes (`Restart=on-failure`). Check its status first:

```
sudo systemctl status tide
```

A healthy result shows "active (running)" and a plausible uptime. "activating (start-pre)" is normal for about three minutes after a boot or restart, because the service deliberately waits 180 seconds before starting tide.py (Section 2.3). If it shows "failed" or "inactive", the service has stopped and needs a restart.

> **CAUTION:** Stations installed before the Restart=on-failure change keep their existing `/etc/systemd/system/tide.service`; install.sh never overwrites an existing service file. On those stations a crash leaves tide.py down until someone restarts it. To check, run `grep Restart /etc/systemd/system/tide.service`. If nothing is printed, escalate so the service file can be updated.

Several other services support tide.py and are worth checking when something looks wrong:

Service	Role, and what fails without it
influxdb3	Local InfluxDB 3 Core database (localhost:8181). Required. Without it, local time-series writes, Grafana, and the cloud sync all fail, although tide.py itself keeps running. Restarts itself automatically (Restart=always).
cloudflared	Cloudflare Tunnel connector. Without it the public website, the Notehub feed for Notecard stations, Grafana, and all remote SSH/VNC access are unreachable from outside the station's own network.
apache2	Serves the station website and the alert-subscription CGI scripts.
grafana-server	Web front end for viewing the local InfluxDB data. Not needed for data collection.
tidemonitor	Headless collector-only alternative to tide. A station runs one or the other, never both: each refuses to start if the other is running. Normally absent or disabled.

The tide plot image is not generated by a service. tideplot.py runs from the tide user's crontab at 1, 21, and 41 minutes past each hour; check with `crontab -l -u tide`.

2.2 Is the Website Actually Updating?

Even if the service shows "active", confirm that the public-facing output is current. Open the station's tide page in a browser (the URL is station-specific; check TIDE_URL in tide.env if unknown) and look at the timestamp of the most recent plotted reading.

What counts as stale depends on how the sensor reports:

- LoRa sensors report about once a minute. A gap of more than a few minutes means something upstream of the display has stopped working.
- Notecard sensors store readings and send them to Notehub in batches, every 15 minutes when the battery is healthy. The newest reading can be up to about 15 minutes old during normal operation. On a low battery the sensor sends less often, eventually only once a day (Section 4.2), so a long gap on a Notecard station may mean a battery problem rather than a software one.

A quick command-line check of the same thing, from the Pi itself:

```
sqlite3 /var/www/html/tides.db "select max(dtime) from sensors;"
```

This is the default database location. The actual path is set by SQL_PATH in tide.env. Compare the timestamp to the current time. A gap larger than the expected reporting interval means the sensor data pipeline has stalled, even if tide.py itself is still running.

If readings are arriving but not appearing, check the log (Section 2.3) for "rejecting outlier" messages. tide.py discards readings that jump more than one foot from the last accepted value (Section 6.3). A run of rejections usually points to a sensor problem, such as an obstruction or a misaligned mount, rather than a software fault. For a history of reporting gaps on LoRa sensors, run:

```
cd /home/tide/bin/tidegauge
/home/tide/.tidenv/bin/python3 find_sensor_gaps.py --hours 24
```

2.3 Common Remedial Actions

Restarting the main service

This is the correct first response to almost any "it's misbehaving but I don't know why" situation. It clears any hung state without needing to diagnose the underlying cause:

```
sudo systemctl restart tide
sudo systemctl status tide   # confirm it came back up cleanly
```

Allow about three minutes before expecting fresh data. The service waits 180 seconds (`ExecStartPre=/bin/sleep 180`) before starting tide.py, on every start and restart, so that networking and other system services have time to come up after a boot.

If the log shows repeated InfluxDB write or query errors, restart the database as well:

```
sudo systemctl restart influxdb3
sudo systemctl status influxdb3
```

Rebooting the Raspberry Pi

If a service restart doesn't resolve the issue, or the Pi itself seems unresponsive (SSH hanging, web server not responding at all), a full reboot is the next reasonable step. Every service in Section 2.1 is enabled at boot, so no manual intervention is required afterward:

```
sudo reboot
```

Checking recent log output

If a restart doesn't resolve the problem, or you want to understand what went wrong before restarting, check tide.py's log:

```
tail -100 /home/tide/bin/tidegauge/newtide.log
```

The log rotates at midnight. The previous day's log is `newtide.log.1` (plain text); older days are compressed as `newtide.log.2.gz` through `newtide.log.7.gz` and can be read with `zcat` or `zless`. Seven days are kept.

Look for repeated exceptions or tracebacks, especially ones right before the service stopped responding. The systemd journal is also worth checking, particularly if a service crashed rather than hung:

```
sudo journalctl -u tide -n 200 --no-pager
sudo journalctl -u influxdb3 -n 200 --no-pager
```

> **NOTE:** Some warnings are expected and do not indicate a fault. For example, an HTTP 429 response during the InfluxDB Cloud sync means the cloud service is rate-limiting; the sync retries automatically on its next 15-minute cycle, and no data is lost.

2.4 Remote Access

Every remote connection to a station goes through its Cloudflare Tunnel (Section 8.4), under subdomains of the station's own domain. No inbound port is open on the station's router.

Access	Use
SSH (ssh.<domain>)	Anything in this section: checking status, restarting services, reading logs. The primary tool for routine remote troubleshooting. The connecting machine needs cloudflared installed and an ~/.ssh/config entry that routes through it, and its public key must be in /home/tide/.ssh/authorized_keys on the station.
VNC (vnc.<domain>)	A full remote desktop, useful for the local Tk display or general desktop-level troubleshooting. On every station the desktop itself is captured and served by TigerVNC (X0tigervnc). What differs is the front end. New stations run RealVNC's service in front of TigerVNC, which adds login with the station's own Unix accounts and TLS encryption. Older stations expose TigerVNC directly (x0vncserver) with a single shared VNC password. Only one front end can hold the VNC port at a time.
Grafana (grafana.<domain>)	Browser access to the station's locally stored readings (Section 6.2). The quickest way to see exactly when data stopped arriving.

NOTE: If cloudflared is down, or the station has lost its internet connection, none of these work, and the public website is unreachable too. Recovery then needs someone on site. Remote access also only reaches the Pi itself: if the problem is with an external account or service (Notehub, Brevo, Twilio, Cloudflare, InfluxDB Cloud; see Section 10), escalate per Section 2.6.

2.5 What NOT To Do Without Escalating

CAUTION: Do not edit tide.env, tide_constants.json, sensor_fields.json, or any of the key files (k1, k2, k3, ku) without understanding their role (Section 7 and Appendix A). These control encryption and station-specific configuration. tide_constants.json is itself encrypted, and an incorrect edit to it or to the keys can silently break alert delivery or, in the worst case, make stored subscriber data unrecoverable.

CAUTION: Do not manually edit the SQLite database (tides.db) unless you are certain of the schema and are working on a backup copy, not the live file. tide.py accesses this file continuously, and an edit at the wrong moment can corrupt it or collide with an in-progress write. Station configuration (the iparams table) in particular controls sensor calibration and which sensor is primary.

CAUTION: Do not delete or move files under /home/tide/influxdb3/, and do not delete `.cloud_sync_watermark` in the tidegauge directory. Without the watermark, the next cloud sync starts 24 hours back, and any older readings that had not yet been sent are never sent.

Routine restarts, reboots, and log inspection (Sections 2.3 and 2.4) are safe to perform without prior authorization. Anything involving configuration files, encryption keys, database contents, or code changes should be escalated first.

2.6 Escalation

For issues that aren't resolved by a service restart or reboot, or that involve any of the caution items above, escalate to the system's maintainer rather than attempting further changes.

Contact: cfi1513840@gmail.com

Include the station's name or domain, what you observed and when, anything you have already tried (Sections 2.3 and 2.4), and any relevant lines from newtide.log or the systemd journal.

3. System Overview

3.1 High-Level Description

Each ACTION station centers on a Raspberry Pi running tide.py, a long-running Python process that:

- receives water-level readings from one to three sensors, over LoRa radio, from Blues Notehub, or read back from InfluxDB Cloud;
- screens each reading for implausible jumps (Section 6.3), then stores it in a local SQLite database and a local InfluxDB 3 Core time-series database;
- generates the station web page (tide.html) every minute and the National Weather Service forecast section every hour, and drives the optional local Tk display;
- checks subscriber alert thresholds and sends email and SMS notifications when they are crossed (Section 9);
- pushes new readings to the shared InfluxDB Cloud database every 15 minutes (Section 6.2).

Apache serves the web pages and a small set of CGI scripts that handle alert subscriptions (sign-up, login, password management). The historical tide plot page is rendered separately by tideplot.py, from cron. Everything reachable from the internet (the website, the Notehub feed, Grafana, SSH and VNC) arrives through the station's Cloudflare Tunnel (Section 8.4); no inbound port is open.

3.2 Sensor Data Paths

A station's sensor data arrives by one of the paths below. Each sensor slot (station 1, 2 or 3) has one type, set by `STATIONn_TYPE` in tide.env: `lora` (Path A), `note` (Path B), `cloud` (Path D), or blank for an empty slot. tide.py reads the types once, at startup. A station can combine `lora` slots with either `note` or `cloud` slots, but not `note` with `cloud`; tide.py refuses to start, with a message saying why, if the types break that rule or aren't valid. Turning a slot on and off is `s1enable–s3enable` in the iparams table, which works like a shut-off valve: a disabled slot's readings are ignored, whatever its type.

Path A: LoRa radio

A Heltec CubeCell transmitter (SampleCubeCellTX sketch, Section 4) takes a burst of readings from the ultrasonic sensor, averages the ones that agree, and transmits a short text packet over LoRa radio roughly once a minute. A Heltec WiFi LoRa 32 receiver (SampleLoRa32RX sketch) adds the received signal strength and passes the packet to the Pi over USB serial (`/dev/ttyUSB0`, set by SERIAL_PORTS in tide.env).

tideget.py's ReadSensor checks each packet on every five-second cycle of tide.py's main loop. It rejects any packet with a malformed field or a missing required field, since a weak radio link can corrupt or truncate a packet. LoRa packets carry only a slot number, so ReadSensor looks up the sensor's three-letter ID in tide.env (`STATIONn_SENSOR_ID`). tide.py then adds the calibration height from iparams and the link type from tide.env before handing the reading on. The serial ports are opened only when a slot is of type `lora`; `SERIAL_PORTS` can be left blank on a station without one.

Path B: Blues Notecard through the Pi

A Blues Notecard paired with a Swan microcontroller (SampleNotecardTX sketch) takes a reading every minute and stores it. It sends the stored readings to Blues Notehub.io over its own cellular connection as a single event, each reading with its own timestamp. It sends every 15 minutes when the battery is healthy, and less often as the battery runs down (Section 4.2). Notehub then forwards the event as an HTTPS POST to the station's `note.<domain>` address. The Cloudflare Tunnel delivers it to tidenote.py's NotehubReceiver, listening only on localhost port 8088.

The receiver accepts the event only if its X-Auth header matches NOTEHUB_SECRET in tide_constants.json. It is checked without blocking on every cycle of tide.py's main loop, so Notehub's delivery timing never holds up the rest of the loop. Each reading in the batch is stored individually, with its original timestamp, and is checked against alerts individually (Section 9.3).

Path C: Blues Notecard directly to InfluxDB Cloud

A Notehub route can instead send a Notecard's events to a Cloudflare Worker, which converts them into InfluxDB records and writes them straight into InfluxDB Cloud, bypassing the Pi entirely. The Worker is configured in Cloudflare and Notehub, not in this repository. A station whose sensor uses this route should have that slot's SnCLOUD_ENABLE set to false in tide.env, so the Pi doesn't send the same readings to the cloud a second time.

Path D: InfluxDB Cloud back to the Pi

A slot of type `cloud` takes its readings from InfluxDB Cloud: readings that some sensor already has there, whether a Notecard sensor written by the Path C Worker or a LoRa sensor synced up by another station. tidecloud.py's CloudReader selects them by the sensor's three-letter ID, so `STATIONn_SENSOR_ID` must be set for every `cloud` slot. This lets a station show and alert on a sensor it has no radio or Notehub link to.

The read never holds up the main loop. At the start of each minute tide.py starts a query on a background thread and carries on; on each five-second cycle it collects any finished results. Each query asks only for readings newer than the newest one already stored for that sensor, so a report is picked up whole however long the sensor waited between reports, and once an hour the query covers the last 24 hours to catch a report that arrived out of order. Every new reading is stored and alert-checked exactly as a Notecard reading is (Path B), in both tides.db and local InfluxDB. Cloud slots are left out of the station's own cloud sync (Section 6.2), since their readings came from there.

> **NOTE:** Paths A, B and D share nothing between the sensor and tide.py: not the hardware, the transport, or the software module. They meet at the same point, DbManage.insert_tide() in tidedatabase.py, and are handled identically from there on. Earlier versions of this manual referred to a SENSOR_SOURCE setting in tide.env, and later to a link type in the iparams table (`s1type-s3type`); both are replaced by `STATIONn_TYPE` in tide.env, and drop_iparams_types.py removes the old iparams columns (Section 5.3).

3.3 From Ingestion to Display

Whichever path delivered it, each reading goes through the same steps:

- **Screening:** a reading at one of its slot's known phantom distances (`STATIONn_PHANTOM_MM`, Section 6.3) is dropped first. Otherwise insert_tide() converts the distance to a tide height using the sensor's calibration and compares it with that sensor's last accepted value (Section 6.3). A rejected reading is logged and stored nowhere.
- **SQLite:** the reading is added to the sensors table in tides.db, which acts as a diagnostic log (Section 6.1).
- **InfluxDB:** the reading is queued for the local InfluxDB 3 Core database and written by a background thread, in batches of up to 500, so a slow database write never delays the main loop or a Notehub response (Section 6.2).
- **Display and web page:** once a minute, tide.py reads the last 24 hours back from InfluxDB, redraws the Tk display if one is running, and regenerates tide.html.
- **Alerts:** readings from the station's primary sensor are checked against subscriber thresholds: once a minute for a LoRa sensor, and for every reading in the batch for a Notecard or cloud sensor (Section 9.3).
- **Cloud:** every 15 minutes, readings not yet sent are pushed to InfluxDB Cloud, except those of cloud slots.
- **Historical plot:** tideplot.py runs from cron at 1, 21 and 41 minutes past the hour. It reads the periodic SQLite copy (SQL_COPY), not the live database, and publishes tideplot.html. The same file also runs as tideplot.cgi when a visitor asks for a custom plot.

3.4 Main Loop Schedule

tide.py's main loop runs every five seconds: under Tk's timer when the local display is enabled, or a plain timed loop when started with the `notk` argument. A counter (main_loop_count) runs from 1 to 12 over each minute, and each slow task is assigned its own count, so tasks never compete within the same five-second cycle.

When	Task
Every cycle (5 s)	Read the LoRa serial port(s); check for queued Notehub events; collect finished InfluxDB Cloud reads (Path D).
Count 2, every 5 min	Fetch current weather from OpenWeatherMap (WX_SERVICE=openwx).
Count 3	Delete alert-subscription registrations not confirmed within 10 minutes.
Count 4, every 30 min	Fetch NDBC buoy and marine observations, keeping each buoy report as its own row in ndbcdata (Section 6.1).
Count 5, 07:00 daily	Email the website visit report to the administrators.
Count 6, once a day	Update sunrise/sunset times and refresh NOAA tide predictions.
Count 7, every 20 min	Copy tides.db to SQL_COPY (at :00, :20 and :40).
Count 8	Send queued email from the mail spool (Section 7.4).
Count 9, every 15 min	Sync new readings to InfluxDB Cloud (at :02, :17, :32 and :47).
Count 12 (once a minute)	Start the next InfluxDB Cloud read for cloud slots; reload iparams; read the last 24 hours from InfluxDB; update the Tk display and tide.html; at the top of the hour, regenerate the NWS forecast; run the out-of-service check; check LoRa alerts.

The out-of-service check covers the primary sensor. If it has not reported for more than 5 minutes, tide.py sends an SMS and email to the administrators. If the other sensor slot (1 or 2) is enabled, it also switches the station to that sensor, updating stationid in iparams.

> **NOTE:** The whole body of the loop is wrapped in an exception handler that logs any error with a traceback and carries on, so one failing task cannot stop the loop from rescheduling itself. A crash of the whole process is handled by systemd instead (Section 2.1).

4. Hardware and Firmware

4.1 Physical Components

Component	Role
MaxBotix ultrasonic sensor (MB7388 or similar)	Measures the distance in millimeters from a fixed mounting point down to the water surface; a smaller distance means a higher water level.
Heltec CubeCell (LoRa path)	Reads the sensor and transmits readings over LoRa radio.
Heltec WiFi LoRa 32 (V3) (LoRa path)	Receives LoRa transmissions and passes them to the Raspberry Pi over USB serial.
Blues Notecard + Swan (cellular path)	Reads the sensor and sends batches of readings to Notehub.io over cellular.
Raspberry Pi (4 or 5)	Runs tide.py and all station software; hosts the web server and local databases.

Tide height is calculated from each sensor's calibration value in iparams (`station1cal` to `station3cal`), the sensor's height in feet above mean lower low water: tide height = calibration − distance ÷ 304.8.

4.2 Firmware Sketches

The repository holds three general-purpose Arduino sketches, one per hardware and data path. Each is a template: station-specific values are set with #define lines or constants at the top of the sketch before it is uploaded.

Sketch	Target hardware	Notes
SampleCubeCellTX.ino	Heltec CubeCell	LoRa transmitter. Set the station number (1–3) and radio frequency; the receiver must use the same frequency. Sends distance (R), battery voltage (V), packet count (C), agreeing-sample count (M), and optional solar voltage (s) and temperature (t).
SampleLoRa32RX.ino	Heltec WiFi LoRa 32 (V3)	LoRa receiver. Adds received signal strength (P) and writes each packet to USB serial.
SampleNotecardTX.ino	Blues Notecard + Swan	Cellular transmitter. Set the three-letter sensor ID (e.g. PRO) and the Notehub product UID. Takes 20 samples a minute and averages those that agree within ±200 mm. Sends a batch every 15 minutes, stretching to 30, 60 and 120 minutes, and finally once a day, as the Notecard's supply voltage falls below 3.90, 3.75, 3.60 and 3.45 V.

A reading's single-letter field codes (S, R, V, M and so on) follow it all the way into InfluxDB; Section 6.2 lists what each one becomes.

> **NOTE:** A Notehub.io account is needed for any station using a Notecard (Section 10). The Notecard does not appear in the Python code; the Pi only ever sees the events Notehub forwards to it (Path B), and a Path C sensor never reaches the Pi at all.

5. Software Architecture

All station-side code is Python: a set of modules imported by tide.py, standalone CGI scripts for the alert-subscription website, and a handful of utility scripts. The tables below summarize each file's role; the source files carry fuller descriptions in their own header comments.

5.1 Core Modules

Module	Role
tide.py	Main process, managed by systemd as the tide service. Runs the five-second main loop and its task schedule (Section 3.4), and owns log rotation (newtide.log). Arguments: `notk` runs without the Tk display, `debug` or `log` raise the log level, and `noaa` forces a refresh of the NOAA predictions at startup.
tidemonitor.py	Headless, collector-only alternative to tide.py: reads LoRa sensors and writes to the databases, with no display, web pages or alerts. Runs as the tidemonitor service. It and tide.py each refuse to start if the other is running.
tidehelper.py	Shared building blocks. Constants loads tide.env, the encrypted tide_constants.json and sensor_fields.json, and creates the InfluxDB clients. Also contains OutlierTracker (Section 6.3), Notify (email, SMS and the mail spool), SunTime, and DailyVisitReport (the daily website visit summary).
tidedatabase.py	DbManage: all reads and writes to SQLite and InfluxDB, including outlier screening on insert, the background InfluxDB write thread, and the cloud sync.
tideget.py	ReadSensor (LoRa serial packets), GetWeather (OpenWeatherMap current weather, and NDBC buoy observations including the buoy's recent report history), and GetNOAA (tide predictions).
tidenote.py	NotehubReceiver: the localhost HTTP listener for Notehub events (Section 3.2, Path B).
tidesensors.py	Sensor slot types: reads `STATIONn_TYPE` from tide.env and checks them at startup, and the shared code that stores and alert-checks a batch of Notecard or cloud readings.
tidecloud.py	CloudReader: reads cloud slots from InfluxDB Cloud on a background thread (Section 3.2, Path D).
tidealerts.py	TideAlerts: checks subscriber thresholds and tracks alert state (Section 9).
tidepredict.py	Turns NOAA's high/low predictions into a one-minute predicted-tide curve.
tideprocess.py	Finds the measured highs and lows used for the high/low markers on the display and web page.
tidehtml.py	Generates tide.html, the station's main tide and weather page.
tidewxhtml.py	Generates the National Weather Service forecast and marine forecast section of tide.html.
tidedisplay.py	The local Tk display, started by tide.py unless the `notk` argument is given.
tideplot.py	Renders the historical plot page (Section 8.1). Runs from cron to publish tideplot.html, and as tideplot.cgi (a symlink) for custom plots on request.
tidecrypto.py	Loads the encryption keys and handles password hashing and verification (Section 7). Symlinked into cgi-bin for the CGI scripts.
tidetime.py	Formats clock times for display (12- or 24-hour, by `12H_TIME` in tide.env) for the Tk display, web pages and alert messages. Has no other project imports, so tideplot.py can use it as a CGI script.

5.2 CGI Scripts (Web-Facing)

These run under Apache's mod_cgid as the www-data user. Apart from sitename.cgi and tideplot.cgi, they implement the alert-subscription workflow. None of them sends email directly; each writes a request to the mail spool for tide.py to send (Section 7.4).

Script	Purpose
alertform.cgi	Registration and login.
valuser.cgi	Confirms a new registration from the emailed link.
processalerts.cgi	Saves a subscriber's alert settings.
changealertpw.cgi	Password change.
forgotpass.cgi, reset-pw.cgi, reset-pw-1.cgi	Forgotten-password request and reset.
sitename.cgi	Supplies STATION_LOCATION for the home page title (Section 8.5).
tideplot.cgi	Symlink to tideplot.py; renders custom historical plots.

NOTE: Python 3.13 removed the cgi and cgitb modules these scripts use. The legacy-cgi package, listed in requirements.txt, restores them. It is a stopgap rather than a permanent fix (Sections 8.3 and 12).

5.3 Installation and Maintenance Utilities

These are run by hand or by install.sh, never by tide.py. Run them from the tidegauge directory with the virtual environment's Python (`/home/tide/.tidenv/bin/python3`).

Script	Purpose
install.sh	Installs a new station or prepares an upgrade. See the Installation Manual.
makekeys.py	Generates the four encryption keys (k1, k2, k3, ku). Run by install.sh on a fresh install only.
encrypt_constants.py / decrypt_constants.py	Encrypt a clear-text constants file, or decrypt tide_constants.json to a temporary clear-text copy for editing. Both use the ku key in their own directory. Delete the clear-text copy after use.
check_config_drift.py	Compares a config file's parameter names against its template and reports missing or obsolete ones. Used by install.sh.
configure_iparams.py	Interactive sensor setup for a new tides.db: which sensors are installed, their calibration, and which is primary. Each sensor's type is set in tide.env, not here. Run by install.sh on a new database only.
find_sensor_gaps.py	Reports gaps in sensor reporting from the SQLite sensors table, grouped by length, for the enabled `lora` slots. Mainly for checking LoRa reception.
drop_iparams_types.py	Removes the unused `s1type-s3type` columns from iparams, keeping every other column in place. Checks that tide.env has all three `STATIONn_TYPE` lines and that they match, refuses while tide.py is running, and backs up tides.db first. Run once per station, with tide.py stopped; `--dry-run` shows what it would do.
visitor_diagnostic.py	Breaks down website visits by host, for looking into something unusual in the daily visit report.
nws_grid_search.py / nws_office_wave_survey.py	Find the NWS forecast grid points to use for NWS_LOCAL_GRIDPOINTS and NWS_MARINE_GRIDPOINTS when setting up a new station.

6. Data Storage

A station keeps its data in three places, each for a different job:

Store	Holds
SQLite (tides.db)	Station configuration, subscriber accounts and alert settings, cached predictions and weather, and a log of every accepted sensor reading. Local to the Pi.
InfluxDB 3 Core	The station's primary time-series store of sensor readings. Local to the Pi; read by tide.py for the display and web page, and by Grafana.
InfluxDB Cloud	A shared copy of every station's readings, and the off-site backup. Fed by each station's cloud sync, and directly by Notehub for Path C sensors.

6.1 SQLite

The SQLite database lives at the path set by SQL_PATH in tide.env, normally `/var/www/html/tides.db`. SQLite suits a single-writer embedded system: no server to run, and the whole database is one file. Its main operational limit is lock contention when several processes use the file at once, which is why the web-facing CGI scripts hand outgoing email to tide.py through a file-based mail spool (Section 7.4) instead of writing to the database themselves.

The table definitions are in `sqltides.db` in the repository, which is also the empty template install.sh copies into place on a new station. Open it with `sqlite3 sqltides.db .schema` to see every column.

Table	Contents
iparams	Station configuration, one row: the primary sensor (stationid), each sensor slot's calibration height (station1cal–station3cal) and enabled flag (s1enable–s3enable), plus debug, tide_only (no weather) and alert-suppression flags. Set up by configure_iparams.py and re-read by tide.py every minute. Each slot's type is in tide.env, not here; older databases may still have unused `s1type-s3type` columns until drop_iparams_types.py removes them.
sensors	Every accepted reading: time, sensor slot, distance (feet and millimeters), signal strength, battery and solar voltage, temperature, agreeing-sample count and packet count. A diagnostic log; the display and web page read from InfluxDB instead.
predicts	NOAA high/low tide predictions, refreshed daily.
wxdata	The latest local weather (OpenWeatherMap).
ndbcdata	NDBC buoy observations, one row per buoy report, keyed by the report's own time. On the first fetch after an upgrade or a new install it is filled with the buoy's recent history (about 45 days). A value the buoy left out of a report is carried forward from its last report in the previous 24 hours. Used for the water temperature plot (Section 8.1) and the water temperature trend in alerts (Section 9.3). Not trimmed (Section 12).
userpass	Subscriber accounts: encrypted email address, hashed password, confirmation status and pending confirmation key.
useralerts	Each subscriber's alert settings: encrypted email and phone number, and thresholds, enable flags and state for each alert type.
banner	Banner message and sunrise/sunset text for the web page.

Every 20 minutes tide.py copies the live database to SQL_COPY (normally `/home/tide/Uploads/tides.db`). tideplot.py reads that copy, and so should any ad hoc query, so that nothing else competes with tide.py for the live file.

6.2 InfluxDB

Local: InfluxDB 3 Core

Each station runs its own InfluxDB 3 Core server as the influxdb3 service (`influxdb3_template.service` in the repository). It listens only on localhost port 8181 and stores its data under `/home/tide/influxdb3/data`. Readings go into the database named by INFLUXDB_LOCAL_DATABASE (normally TideData), in the measurement tide_station. The access token, INFLUXDB_LOCAL_TOKEN, is in tide_constants.json.

Writes and reads use two different client libraries:

- Writes use InfluxDB 3's v2-compatible write endpoint (the influxdb-client package), from a background thread in DbManage. The thread writes whatever has queued up in batches of up to 500 points, so a backlog (a Notecard batch, a cloud catch-up) is written in a few requests rather than one per reading; if a batch fails it falls back to writing those points one by one. A native InfluxDB 3 write path was tried and abandoned: on one station it silently failed to store data. The background thread means a slow write can never stall the main loop or a Notehub response.
- **Reads** use SQL through the influxdb3-python package, since InfluxDB 3 does not support the older Flux query language. Query results are converted with pandas, which keeps InfluxDB's nanosecond timestamps intact.

> **NOTE:** Some stations upgraded from InfluxDB v2 still have the old v2 server installed, with its historical data, on port 8086. tide.py no longer uses it. It is kept only so that the older data can still be queried.
>
> **NOTE:** InfluxDB 3 Core refuses a query that would read more than 432 data (Parquet) files, reporting "Query would scan N Parquet files, exceeding the file limit". The limit is roughly three days of data. tide.py's own queries stay within it; an ad hoc query over a longer period should be split into shorter time ranges.

Reading fields

`sensor_fields.json` maps each single-letter code in a reading to its InfluxDB name and says whether it is stored as a tag (used to filter and group) or a field (a measured value). Every point also carries two tags from tide.env: location (INFLUXDB_LOCATION) and sensor_type (INFLUXDB_SENSOR).

Code	InfluxDB name	Type	Meaning
S	sensor_num	tag	Sensor slot, 1–3
I	sensor_id	tag	Three-letter sensor ID, e.g. PRO, BEL, DAM
L	link_type	tag	lora or note
H	sensor_height_ft	field	Calibration height above MLLW, feet
R / U	sensor_measurement_mm	field	Distance to the water, millimeters
M	correlation_count	field	Samples that agreed, out of those taken
C	message_count	field	Packet counter (LoRa)
V	battery_milliVolts	field	Battery voltage, millivolts
P / r	signal_strength	field	Received signal strength
s	solar_milliVolts	field	Solar panel voltage, millivolts
t	temperature	field	Temperature sensor reading

> **CAUTION:** A column that has been written as a tag cannot later be written as a field, or the reverse; InfluxDB rejects the write outright. Change a code's type in sensor_fields.json only with a plan for the existing data, locally and in the cloud.

Cloud: InfluxDB Cloud Serverless

All stations share one InfluxDB Cloud organization and bucket (INFLUXDB_CLOUD_ORG and INFLUXDB_CLOUD_BUCKET in tide.env, normally TideGauge and TideData). This is the only place readings from different stations can be compared, and it is each station's off-site backup.

Every 15 minutes, sync_influxdb_cloud() reads local readings newer than a saved timestamp, the watermark, and writes them to the cloud:

- The watermark is kept in `.cloud_sync_watermark` in the tidegauge directory, so it survives restarts. Each sync reads at most 12 hours past the watermark, keeping the query well within InfluxDB 3 Core's file limit; a larger backlog is worked through one 12-hour window per sync. A window with no readings, ending more than an hour ago, just moves the watermark on.
- Without a watermark file (a new station, or one deleted by hand), the sync starts 24 hours back. A watermark more than 7 days old jumps forward to 7 days back, with a warning in the log; readings older than that are not sent.
- Readings go in batches of 500 with a short pause between batches, to stay within InfluxDB Cloud's rate limits. The watermark moves forward after each successful batch, so an interrupted sync resumes where it stopped.
- Which columns are tags is worked out from sensor_fields.json each time, so the cloud copy always matches the local one.
- Sensor slots with `SnCLOUD_ENABLE=false` in tide.env are skipped. Use this for a Path C sensor, whose readings already reach the cloud directly from Notehub. Cloud slots (Path D) are always skipped, and when no slot is left to send, the sync does nothing.

The sync never delays anything else: if the cloud is unreachable or returns an error such as HTTP 429 (rate limited), it logs a warning and tries again 15 minutes later.

Viewing the data

Grafana (the grafana-server service, reached at `grafana.<domain>`) is the standard way to view and graph a station's local data, using an InfluxDB datasource in SQL mode pointed at localhost:8181. InfluxDB 3 Core has no built-in web interface. InfluxData's separate InfluxDB 3 Explorer would also work, but Grafana is more capable and easier to use, so it is what every station runs.

Readings that were stored but are known to be wrong, such as a run of phantom readings recorded before `STATIONn_PHANTOM_MM` was set for the sensor (Section 6.3), can't be deleted from InfluxDB: row deletion is an InfluxDB 3 Enterprise feature, not available in Core, and InfluxDB Cloud Serverless doesn't support deleting data at all. Hide them in Grafana instead by adding a condition to the panel's SQL query. For example, to leave out sensor DAM's readings between 1145 and 1170 mm:

```
AND NOT (sensor_id = 'DAM' AND sensor_measurement_mm BETWEEN 1145 AND 1170)
```

The same condition works for a Grafana panel reading InfluxDB Cloud, since the cloud copy has the same columns. The rows themselves stay in both databases. Locally they drop out of tide.html and the Tk display after 24 hours; in the cloud they are removed only by the bucket's retention period. In SQLite, where the historical plot (tideplot.py) reads from, such rows can simply be deleted (Section 6.3).

6.3 Outlier Rejection

Ultrasonic sensors occasionally report a false distance, from a wave, a bird, spray or a radio error. Every reading is screened before it is stored, and screened again before it is used for alerts, by the same OutlierTracker class in tidehelper.py. There is one tracker per sensor ID for storage (in tidedatabase.py) and one for the primary sensor's alerts (in tidealerts.py), so a fix to the logic only ever has to be made once.

- **Baseline:** the tracker keeps a single baseline value, the last accepted tide height. A new reading within 1 foot of it is accepted and becomes the new baseline. Anything further away is rejected and logged as "rejecting outlier ... versus baseline ...".

- **Establishing a baseline:** at startup, and after a reset, the tracker needs 5 readings in a row, each within 1 foot of the one before, before it has a baseline. Readings during this phase are all accepted. If one disagrees, the count restarts from that reading.
- **Gap reset:** if more than 5 minutes pass without an accepted reading, the baseline is discarded and a new one established. This covers both a real reporting gap and a sensor stuck reporting rejected values, where the old baseline is no longer a fair comparison.

Readings are judged by their own timestamps. For a Notecard batch, each reading's own time is used rather than the time the batch arrived, so the roughly one-minute spacing within a batch and the gap between batches are both seen correctly. LoRa readings carry no timestamp and use the Pi's clock.

> **NOTE:** This replaced an earlier 20-sample rolling average, which could stay stuck rejecting good readings for long periods after a gap, and existed as two separate copies that had drifted apart. A reading is screened only when its sensor slot has a calibration height set in iparams; until then it is stored unscreened.

Phantom readings

Some installations return a false echo at a fixed distance, a reflection off the dock structure, for hours at a time at certain tide levels. At dscottatide the DAM sensor read 1154–1161 mm (about 16.2 ft) for seven hours one night while the real tide was 2–5 ft. A phantom agrees with itself, so the outlier screen can't tell it from the tide: if it arrives while a baseline is being established, it becomes the baseline, and the real readings are then rejected as outliers.

`STATIONn_PHANTOM_MM` in tide.env lists a slot's known phantom distances, comma-separated: a single value in mm, matched within 10 mm, or a range such as `1145-1170`. tidesensors.py drops a reading at one of those distances before anything else sees it: it is not stored, not outlier-screened and not checked for alerts, on the LoRa, Notecard and cloud paths alike. The log records when a run of phantom readings starts, and its count and range when a real reading arrives. A value that is not a number or a range stops tide.py at startup with a message. While a phantom lasts, the slot simply has no readings.

To find a phantom's distance, look at the raw distances stored over the bad stretch; a phantom shows as the same distance, give or take a few mm, reading after reading:

```
sqlite3 /var/www/html/tides.db "select dtime, distmm, nbrsams from sensors where
stationid=2 and dtime > '2026-11-01' order by dtime"
```

Phantom readings stored before the setting was added can be removed from SQLite (back up tides.db first), which also clears them from the historical plot at the next 20-minute copy:

```
sqlite3 /var/www/html/tides.db "delete from sensors where stationid=2 and distmm between
1145 and 1170"
```

Use the slot's own number for stationid and its own phantom range. They can't be removed from InfluxDB; hide them in Grafana instead (Section 6.2, Viewing the data).

7. Security and Encryption

7.1 Overview

Three kinds of subscriber data are protected at rest, each by the method that suits whether the original value is ever needed again:

Data	Mechanism	Why
Email address	Fernet encryption (key k1)	Must be recovered to send alert email.
Phone number	Fernet encryption (key k2)	Must be recovered to send alert SMS.
Password	argon2id hash (one-way)	Only ever needs checking, never recovering, so a copy of the database never reveals a working password.

The station's own secrets (email and SMS account credentials, API keys, InfluxDB tokens and the Notehub secret) are stored in tide_constants.json with every value encrypted by a fourth key, ku (Section 7.5).

7.2 Key Management

Key	Use
k1	Encrypts subscriber email addresses.
k2	Encrypts subscriber phone numbers.
k3	Only for checking passwords stored under the old Fernet scheme and migrating them (Section 7.3). Kept because there is no way to tell from the database alone whether any old-style passwords remain.
ku	Encrypts the values in tide_constants.json. Used only by tide.py and the install-time scripts; the CGI scripts never read it.

makekeys.py generates all four keys once, when a station is first installed. They live only in the tidegauge directory (`/home/tide/bin/tidegauge/`), never in the web server's document root. tidecrypto.py is the single place that loads k1–k3; it is symlinked into cgi-bin so the CGI scripts and tide.py share one copy.

> **CAUTION:** Losing or regenerating the keys makes every encrypted subscriber email address and phone number, and tide_constants.json, permanently unreadable. Back them up with the station's configuration, and copy them, with `cp -p`, whenever a station is upgraded into a new directory. install.sh refuses to generate new keys when it finds existing encrypted data without them.

The key files should be readable by the tide user and, for k1–k3, the tide group, which the web server's www-data account belongs to, and by no one else. To check:

```
ls -l /home/tide/bin/tidegauge/k?
```

Expect `-rw-r-----` (640) for k1–k3 and `-rw-------` (600) for ku, all owned by tide:tide. makekeys.py creates the keys with these permissions, and install.sh re-applies them on every run. Keys generated before that change may still be world-readable (644); re-running install.sh, or the same `chown` and `chmod` commands by hand, corrects them.

7.3 Password Hashing and Legacy Migration

Passwords are hashed with argon2id (the argon2-cffi package), the current OWASP recommendation for password storage. tidecrypto.py's verify_password() accepts both current hashes and older Fernet-encrypted passwords, telling them apart by the `$argon2id$` prefix every hash begins with, which can never appear in Fernet output.

When an old-style password is verified, the calling script immediately replaces it with a hash, so each subscriber is migrated silently the next time they log in, with no forced reset.

7.4 Mail Spool

The CGI scripts never send email or hold email credentials. When one needs to send a message (a registration confirmation, a password reset, or a notice to the administrators), it writes a small JSON file to the mail spool directory, `/var/www/html/mailspool/`. Once a minute, tide.py sends whatever is waiting there using the credentials in tide_constants.json. A message addressed to "ADMIN" goes to the administrator addresses, which the CGI scripts never see.

A message that fails is retried on each of the next two passes. After 3 failed attempts it is moved to `mailspool/failed/` and a warning is logged. Files there can be inspected, and moved back to `mailspool/` to try again once the cause is fixed. Successfully sent messages are deleted.

7.5 Station Secrets and Network Exposure

tide_constants.json holds every credential the station uses, each value encrypted with ku: administrator contacts, Brevo SMTP credentials, Twilio credentials, the OpenWeatherMap API key, both InfluxDB tokens, and NOTEHUB_SECRET. tidehelper.py decrypts it once at startup. To change a value, run decrypt_constants.py to make a clear-text copy, edit it, re-encrypt it with encrypt_constants.py, and delete the clear-text copy; or re-run install.sh, which offers to do the same thing.

Nothing on the station is exposed directly to the internet:

- The router has no open inbound ports. The website, Notehub feed, Grafana, SSH and VNC all arrive through the outbound Cloudflare Tunnel (Section 8.4).
- InfluxDB (port 8181) and the Notehub receiver (port 8088) listen only on localhost; they are reachable from outside only through the tunnel's own routes.
- The Notehub receiver accepts an event only when its X-Auth header matches NOTEHUB_SECRET, and answers anything else with HTTP 401.
- tide.env, tides.db and the mail spool live in the web root, where the CGI scripts can reach them. An Apache rule installed by install.sh (tide-protect.conf) refuses to serve them, or any .json, .db, .log, .tmp, .dev, .bak or .gz file there, over HTTP; a request for https://<domain>/tide.env should return 403.
- SSH accepts only the public keys listed in `/home/tide/.ssh/authorized_keys`. New stations run VNC behind RealVNC with the station's own Unix logins; older stations use a single shared VNC password (Section 2.4).

8. Web Presentation Layer

8.1 Pages

Apache serves everything from its document root, HTML_DIRECTORY in tide.env (normally `/var/www/html/`). install.sh copies the static pages there from the repository; the rest are generated on the station.

Page	Source
index.html	Static. The station home page (Section 8.5).
alertlogin.html, forgotpass.html	Static. Entry points for the alert-subscription workflow.
alert_tutorial.html	Static. A guide for subscribers, linked from the home page.
tide.html	Generated by tidehtml.py every minute: current tide and weather, the tide curve with high/low markers, and the NWS forecast section from tidewxhtml.py (refreshed hourly). Times follow `12H_TIME` in tide.env (2:05 PM, or 14:05 when false); dates are unchanged.
tideplot.html	Generated by tideplot.py from cron at :01, :21 and :41: the historical plot page. Custom plots are drawn on request by tideplot.cgi. The page has the end date, the span in days and Refresh; everything else is in a Plot Options drop-down that opens over the top of the plot: Sensor 1–3, Predicted and Tide Markers (Tide), Wind, Rain, Air Temp and Water Temp (Weather), and BatV 1–3 (Battery). The tide panel is left out when no sensor and no prediction is selected, and its legend lists only what is shown. Water temperature comes from ndbcdata.

Every page shares the same navigation buttons: Home, Request Alerts, Tide & Weather, and Historical Analysis.

Figure 1, at the end of this section, shows a complete tide.html page. From the top: current local weather (OpenWeatherMap) and offshore conditions (NDBC buoy); the measured tide (blue) plotted against the NOAA prediction (grey), with the high and low tide markers and the current reading; the National Weather Service five-day forecast, detailed forecast and radar image; and the hourly marine point forecast. The image is also in the repository as docs/tide_page_full.png.

8.2 CGI Scripts

See Section 5.2 for the list and purpose of each script. All of them run under Apache's mod_cgid as the www-data user, using the station's Python virtual environment (`/home/tide/.tidenv/bin/python3`, named in each script's first line). They read their settings from tide.env through the symlink `/var/www/html/tide.env`, and load encryption keys k1–k3 through the tidecrypto.py symlink in cgi-bin. They never read tide_constants.json or ku.

install.sh creates these symlinks and re-points them if the tidegauge directory moves, which is why it must be run again after an upgrade's directory rename. A broken symlink usually shows up as a CGI script failing with "No module named 'tidecrypto'" in Apache's error log (`/var/log/apache2/error.log`).

8.3 Python 3.13 Compatibility

> **CAUTION:** Python 3.13 removed the cgi and cgitb standard-library modules (PEP 594), which every CGI script and tideplot.py still import. The legacy-cgi package, pinned in requirements.txt, provides a compatible replacement and is installed into the virtual environment by install.sh. It works, but it is a stopgap rather than a permanent fix (Section 12).

8.4 Cloudflare Tunnel

Stations sit on ordinary home or office networks, typically behind a router with no static public IP and no safe way to forward inbound ports. A Cloudflare Tunnel works the other way round: the cloudflared service on the Pi opens

an outbound connection to Cloudflare, and Cloudflare sends public requests for the station's domain back down that connection. No inbound port is opened, and nothing depends on the router's configuration.

Each station has its own domain, registered through Cloudflare, and a tunnel with these routes:

Hostname	Type	Local service	Use
<domain>, www.<domain>	HTTP	localhost:80	Public website (Apache)
ssh.<domain>	SSH	localhost:22	Remote administration
vnc.<domain>	TCP	localhost:5901	Remote desktop
grafana.<domain>	HTTP	localhost:3000	Viewing the station's data
note.<domain>	HTTP	localhost:8088	Notehub feed; Notecard stations only

SSH and VNC through the tunnel need cloudflared on the connecting computer as well; for SSH, an entry in that computer's ~/.ssh/config routes the connection through it. The account, domain and tunnel setup are covered in the Installation Manual's Cloudflare tutorial (prerequisite 6).

> **NOTE:** If the cloudflared service stops, or the station loses its internet connection, every route above stops working at once, including the website. Check `sudo systemctl status cloudflared` on the Pi.

8.5 Site-Specific Customization

index.html and alertlogin.html are identical on every station. Only three things vary per site, each supplied separately, so a new station never needs the shared HTML, CSS or JavaScript edited.

webimage.png

The station's own photo, shown on both index.html and alertlogin.html. install.sh installs a placeholder graphic (webimage.png.template) on a new station; replace it with a real photo. Every page that shows it uses the same 8:5 (1.6:1) aspect ratio: 480x300 on alertlogin.html and 560x350 on index.html. Because of that shared ratio, the browser only ever scales the image evenly, never crops or stretches it.

> **CAUTION:** This only works because every page keeps the same 8:5 ratio. A browser stretches an image to fit whatever width and height a page gives it; it doesn't crop or letterbox on its own. Any new page that shows webimage.png must either use the same 8:5 ratio, or set CSS object-fit (cover to crop, contain to letterbox).

Supply the photo at 960x600 pixels, twice the display ratio, so it stays sharp on high-resolution screens. If the original isn't 8:5, crop it to 8:5 before saving it as webimage.png.

webinfo.txt (optional)

A short plain-text description of the station, shown in an information block on index.html. install.sh installs a generic version (webinfo.txt.template); replace it with the station's own text. If `/var/www/html/webinfo.txt` is missing, the block simply doesn't appear: index.html fetches the file when the page loads and fails silently if it isn't there.

STATION_LOCATION (tide.env)

If set, this tide.env value, also used in email subjects, SMS messages and the Tk display's title, is read by sitename.cgi and put at the front of index.html's title: for example, "Belfast Harbor Tide Station Hub" instead of "Tide Station Hub". If it is unset, or sitename.cgi isn't installed, the page falls back to the generic title.

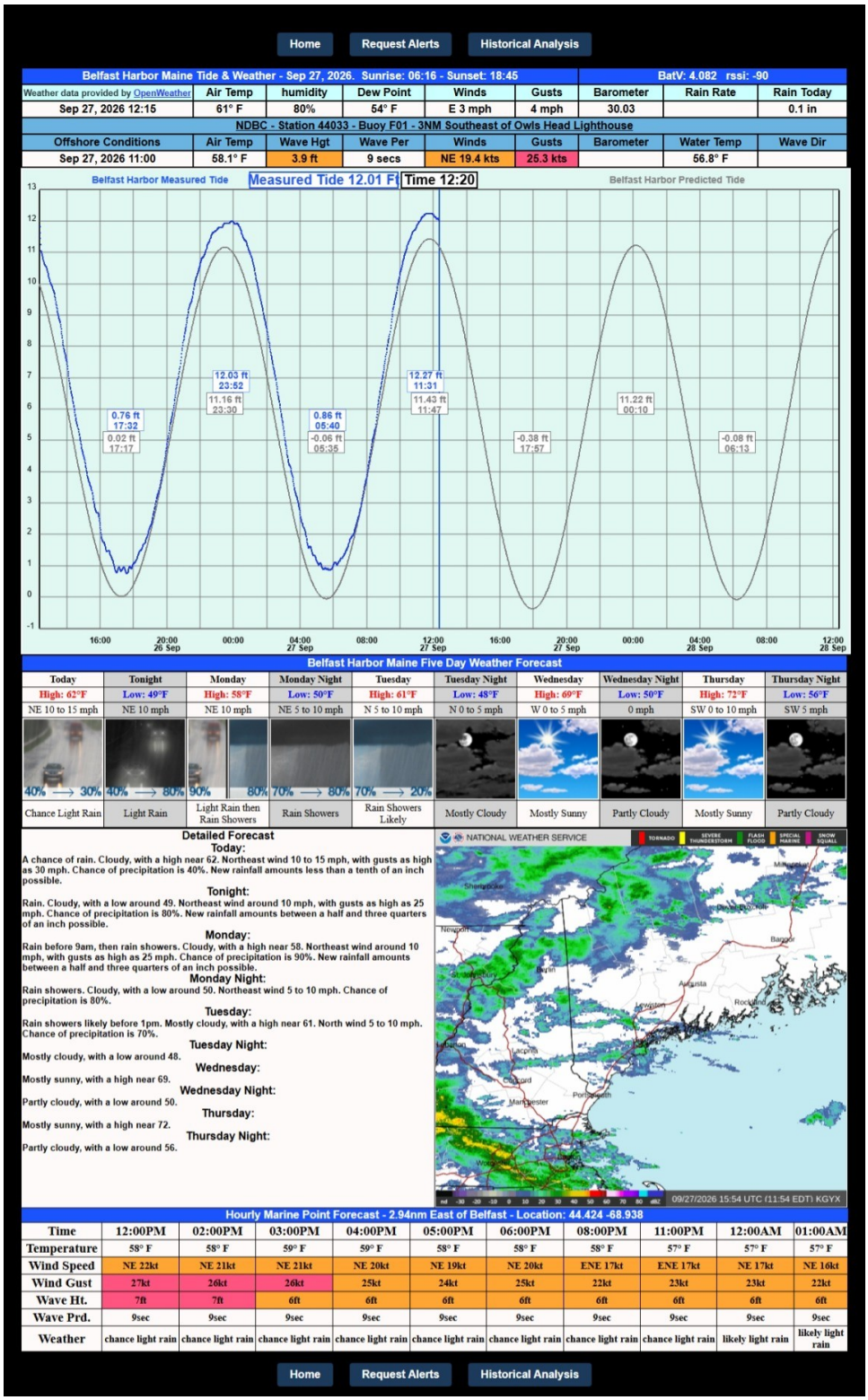


Figure 1. A complete tide.html page, as generated by tide.py (Belfast Harbor, September 27, 2026).

9. Alert System

Anyone can register on the station website for alerts, choosing which conditions to be told about. Alerts go by email, and also by SMS if the subscriber gave a phone number. The station also uses the same channels to send its administrators startup notices, sensor out-of-service warnings, and the daily visit report.

9.1 Email Delivery

Email is sent through Brevo's SMTP relay, the only email service the station uses, with smtplib from Python's standard library and STARTTLS on SMTP_PORT (tide.env, normally 587). The Brevo SMTP server, login and sending address are in tide_constants.json. Every message is sent from the Brevo sending address (BREVO_EMAIL_ADDRESS), which also identifies the station to the National Weather Service's forecast service.

Email is always sent from tide.py, never from a CGI script: alerts directly, and website messages through the mail spool (Section 7.4).

9.2 SMS Delivery

SMS messages are sent through a Twilio account using the twilio package's REST client (tidehelper.py). The account SID, auth token and sending number are in tide_constants.json. Each message is prefixed with STATION_LOCATION, since SMS has no subject line to show which station sent it. Administrator numbers are ADMINBRS1 and ADMINBRS2 in tide_constants.json.

9.3 Alert Evaluation

tidealerts.py's TideAlerts class checks every subscriber's settings (the useralerts table) against the latest tide reading, weather and buoy data. The alert types are:

Alert	Fires when
Tide level	The tide crosses the subscriber's level, rising or falling.
Tidal variation	At the predicted high tide, the measured tide is above the prediction by at least the subscriber's amount (a positive setting), or at the predicted low tide, below it by at least that amount (a negative setting).
High/low tide event	A predicted high or low tide is a set number of minutes away. Can be limited to highs above, or lows below, a given level, and repeats for a set number of tides.
Wind speed	The wind speed or gust, whichever is higher, exceeds the subscriber's value, optionally only from one direction. A follow-up notice is sent when it drops back.
Air temperature	Air temperature reaches the subscriber's value. Re-armed once it has moved 2.5 °F away.
Water temperature	Water temperature (from NDBC) reaches the subscriber's value. Re-armed once it has moved 1 °F away. The message says whether the water is rising, falling or steady, judged day over day: each hour of the last 24 is compared with the same hour the day before, from the ndbcdata report history, so the daily warm-and-cool swing doesn't count as a trend.

Each alert can be enabled or disabled, and limited to daylight hours (between sunrise and sunset). Each subscriber also has a master switch that turns all their alerts on or off. TideAlerts remembers each alert's state between checks, so a crossing sends one notification rather than one on every reading while the condition lasts.

Only readings from the station's primary sensor (stationid in iparams) are checked. How often depends on its link:

- **LoRa:** once a minute, using that minute's reading.

- **Notecard:** once for every reading in each batch as it arrives, each with its own timestamp, so a threshold crossed partway through a 15-minute batch is not missed.
- **Cloud:** the same as a Notecard sensor, for every reading each read from InfluxDB Cloud brings in.

Before any alert is considered, the reading passes through its own OutlierTracker (Section 6.3), separate from the one that screens storage. This tracker resets whenever the primary sensor changes, whether by hand or by the automatic failover in Section 3.4, because two sensors' readings may not agree within the 1-foot window. Accepted readings also feed a rising/falling indicator that compares the averages of the last two groups of 10 readings.

10. External Services and Accounts

A station depends on the outside services below. The accounts can't be created by installing software; each needs its own sign-up, and the Installation Manual's prerequisite tutorials cover them. Credentials are stored encrypted in tide_constants.json unless noted otherwise.

Service	Used for	Notes
Cloudflare	Domain registration; tunnel for all remote access (Section 8.4)	One domain and tunnel per station. Configured in Cloudflare's dashboard and cloudflared on the Pi; no credentials in tide_constants.json.
Blues Notehub.io	Notecard sensor data (Section 3.2, Paths B and C)	Notecard stations only. Charged per event; set up automatic renewal. NOTEHUB_SECRET must match the route's X-Auth header.
InfluxDB Cloud (Serverless)	Shared, off-site copy of every station's readings (Section 6.2)	One organization and bucket shared by all stations; the token is usually issued by the network administrator.
Brevo	Email (Section 9.1)	SMTP relay login and sending address.
Twilio	SMS (Section 9.2)	Account SID, auth token and sending number.
OpenWeatherMap	Current local weather, every 5 minutes	API key (WXOPENAPI). The only weather service in use; the code for WeatherUnderground and WeatherLink remains but is not used.
NOAA CO-OPS	Tide predictions for NOAA_STATION, daily	Public; no account.
NOAA NDBC	Buoy and marine observations for NDBC_STATIONS, every 30 minutes	Public; no account.
National Weather Service (api.weather.gov)	Local and marine forecasts, hourly	Public; no account. Grid points set in tide.env.
GitHub	Source code (github.com/cfi1513840/tide-gauge)	Public repository; needed to install or upgrade.

NOTE: If a service fails, tide.py keeps running and logs the error. Weather, buoy and forecast data go stale until the service comes back, and the cloud sync catches up on its next cycle. Website email waiting in the mail spool is retried, but an alert email or SMS that fails is logged and not re-sent. The exception is Cloudflare: while the tunnel is down, the station can't be reached from outside at all (Section 8.4).

11. Deployment and Installation

The ACTION Installation Manual (`ACTION_Installation_Manual.md` in the repository) is the authoritative guide to building a new station and upgrading an existing one, with a numbered tutorial for each prerequisite account and service. This section only summarizes what the installation does and the rules that matter when maintaining a station.

11.1 install.sh

install.sh is run from the directory the repository was cloned into, as the tide user and never with a blanket `sudo`; it calls `sudo` itself where needed. It is safe to run again: each step checks what is already in place and skips, updates or offers to edit it rather than overwriting it. In outline, it:

- refuses to start if an upgrade directory is missing any per-station file, or if encryption keys are missing while encrypted data exists (Section 7.2);
- installs Apache, SQLite and python3-venv if missing, and creates the Python virtual environment (`/home/tide/.tidenv`) from requirements.txt;
- creates or checks tide.env, tide_constants.json and sensor_fields.json against their templates, backing up and opening an editor when anything differs, and offering to edit them anyway when nothing does;
- installs the tide systemd service (only if none exists), generates the keys on a new station, and sets the key file permissions;
- copies in a new tides.db and runs the sensor setup wizard (only if none exists);
- creates the mail spool directories and the cgi-bin and web-root symlinks, copies the web files and CGI scripts, and adds the tideplot.py cron entry.

install.sh does not yet set two permissions tideplot.cgi needs: Apache must be able to pass through /home/tide (`chmod 711 /home/tide`), and /var/www/html/tideplot.log must be writable by the tide group (owner tide:tide, mode 664). The Installation Manual (section 4.4) has the commands.

11.2 Python Package Requirements

requirements.txt lists every pip-installed dependency, each annotated with which module needs it and why. install.sh installs them into `/home/tide/.tidenv`, and every service, cron job and CGI script uses that environment's Python. The main ones are:

Package	Used for
cryptography, argon2-cffi	Fernet encryption and password hashing (Section 7).
python-dotenv	Loading tide.env.
influxdb-client	All InfluxDB writes, local and cloud.
influxdb3-python, pandas	InfluxDB 3 SQL queries and converting their results. influxdb3-python brings in pyarrow, which takes a long time to install on a Pi.
legacy-cgi	Replaces the cgi module removed in Python 3.13 (Section 8.3).
twilio	SMS.
pyserial	LoRa receiver input.
requests, urllib3, wget, feedparser	Weather, buoy, forecast and prediction downloads.
pytz	Time zone handling.
suntime, suntimes	Sunrise and sunset, in tidehelper.py and tideplot.py respectively (Section 12).

11.3 Upgrading a Station

A running station is upgraded without touching its live directory until the last moment. The Installation Manual (section 3.15) has the full procedure; in outline:

- Clone the new code into `~/bin/tidegauge-new`.
- Copy the per-station files into it with `cp -p`: tide_constants.json, tide.env, sensor_fields.json, k1, k2, k3, ku and .cloud_sync_watermark.
- Run install.sh in the new directory to reconcile the configuration with the new templates.
- Stop the tide service, then rename `tidegauge` to `tidegauge-save` and `tidegauge-new` to `tidegauge`.
- Run install.sh once more from the final directory to re-point the symlinks, then start the service.

To roll back, stop the service and reverse the two renames.

> **NOTE:** install.sh never replaces an existing /etc/systemd/system/tide.service, so an upgrade does not pick up changes to tide.service.template. In particular, stations installed before the Restart=on-failure change need those two lines added by hand, followed by `sudo systemctl daemon-reload` (Section 2.1).

11.4 Source Branches

The repository has two working branches. dev is the stable, production branch that operational stations run. exp is where development happens: changes are made on exp, tested on a test station, merged into dev, and then rolled out to the operational stations.

12. Known Technical Debt

These are known, understood limitations. None is urgent, but each is recorded here so it isn't rediscovered from scratch later.

cgi.FieldStorage() and the Python 3.13 removal

See Section 8.3. legacy-cgi is a working stopgap, not a permanent fix. A proper migration would replace cgi.FieldStorage() with urllib.parse.parse_qsl() for GET and HEAD requests, and the standard library's email.message module (or the multipart package) for POST, following Python's own migration guidance.

SQL built from strings in processalerts.cgi

Several statements in processalerts.cgi, and a few elsewhere, are built by inserting values directly into the SQL text instead of passing them as parameters. The script is only reached after login, and most values are converted to numbers first, but the statements should be audited and converted to parameterized queries (? placeholders), as alertform.cgi already does for its inserts.

Hardcoded time zone

tidedatabase.py and tidealerts.py use `US/Eastern` directly instead of TIME_ZONE from tide.env. At a station outside Eastern time, stored times and the daylight-only alert check would be wrong.

Failed alerts are not retried

An alert email or SMS that fails to send is logged and dropped. Only website email, which goes through the mail spool, is retried (Section 7.4). That may be acceptable, since a late alert has limited value, but it should be a deliberate choice.

Automatic failover needs updating for three sensors

The out-of-service check (Section 3.4) dates from before the third sensor slot was added, and still only switches between slots 1 and 2. A station whose primary sensor is in slot 3 gets the out-of-service notice but no automatic switch, and a slot 3 sensor is never chosen as the replacement. This needs reworking to handle all three slots.

Duplicate sun-calculation packages

Two separate PyPI packages, suntime (used in tidehelper.py) and suntimes (used in tideplot.py), provide overlapping sunrise and sunset calculations. This is almost certainly accidental duplication and a good candidate for consolidating onto one package.

Inactive code retained deliberately

The WeatherUnderground and WeatherLink weather sources, their tide.env settings (WX_UND_*, WX_LINK_*) and their tide_constants.json keys are deactivated in place rather than removed, so they can be brought back if needed; OpenWeatherMap is the only source in use. tidehelper.py also still reads the obsolete SENSOR_SOURCE setting, which nothing uses (Section 3.2).

ndbcdata is not trimmed

ndbcdata keeps every NDBC buoy report it receives (Section 6.1), adding up to about 144 rows a day at a buoy that reports every 10 minutes. That is small, but it grows without limit; a retention period, such as deleting reports older than a year, may be wanted later.

tidemonitor.py and sensor types

tidemonitor.py, the headless collector, has not been updated for sensor types in tide.env (`STATIONn_TYPE`), cloud slots or phantom filtering (`STATIONn_PHANTOM_MM`). It reads LoRa sensors only, as before.

Outlier baseline can lock onto a steady false reading

While the outlier tracker establishes a baseline (Section 6.3) it accepts every reading, and any 5 that agree become the baseline. A steady false reading arriving at that moment is accepted, and the real readings after it are rejected until the next gap reset. Phantom filtering covers the known case. A more general fix would also require baseline candidates to fall within a few feet of the NOAA predicted tide.

13. Appendices

Appendix A -- tide.env Reference

tide.env holds a station's non-secret settings; tide.env.template in the repository is the reference copy, with comments. Secrets are in tide_constants.json instead (Section 7.5).

Variable	Purpose
STATION_LOCATION / STATION_LATITUDE / STATION_LONGITUDE / STATION_NAME	Station name and position: used for sunrise/sunset, page and email titles, and SMS prefixes.
NDBC_LOCATION / NDBC_LATITUDE / NDBC_LONGITUDE	Label and position for the marine point forecast shown on tide.html. (The NDBC prefix is historical; these are not buoy settings.)
NDBC_STATIONS / NDBC_URL / NDBC_TITLE	NDBC buoy used for marine observations, and the link and title shown for it.
NOAA_STATION / NOAA_STATION_NAME	NOAA station for tide predictions, and its label.
NWS_LOCAL_GRIDPOINTS / NWS_MARINE_GRIDPOINTS / NWS_RADAR	National Weather Service forecast grid points and radar image.
WX_SERVICE / WX_OPEN_URL	Weather source (openwx, OpenWeatherMap) and its API address. WX_UND_* and WX_LINK_* belong to inactive sources (Section 12).
SMTP_PORT	SMTP port for the Brevo relay, normally 587 (Section 9.1).
SQL_PATH / SQL_COPY	Live database, and the copy refreshed every 20 minutes for plotting and queries.
HTML_DIRECTORY / CGI_DIRECTORY / CGI_URL / TIDE_URL	Web root, CGI directory, public CGI address, and the public tide page address used in alert messages.
HOME_DIRECTORY	The tidegauge directory, normally /home/tide/bin/tidegauge.
SERIAL_PORTS / USB0_BAUDRATE	Serial port(s) for the LoRa receiver, and speed. Leave `SERIAL_PORTS` blank on a station with no `lora` slot.
STATIONn_NUM / STATIONn_TYPE / STATIONn_SENSOR_ID / STATIONn_LOCATION (n = 1–3)	For each sensor slot: its type (`lora`, `note`, `cloud`, or blank for none; Section 3.2), its three-letter sensor ID, and a location label. The ID tags LoRa readings and selects a slot's readings for Notecard (when more than one `note` slot) and cloud slots.
STATIONn_PHANTOM_MM (n = 1–3)	Known phantom (false echo) distances for each slot's sensor, in mm: single values (±10 mm) or ranges like `1145-1170`, comma-separated. Readings at those distances are dropped. Usually blank (Section 6.3).
SnCLOUD_ENABLE (n = 1–3)	Whether each slot's readings are sent to InfluxDB Cloud by the station (false for a Path C sensor).
INFLUXDB_LOCAL_URL / INFLUXDB_LOCAL_DATABASE	Local InfluxDB 3 Core address and database.
INFLUXDB_CLOUD_URL / INFLUXDB_CLOUD_ORG / INFLUXDB_CLOUD_BUCKET	InfluxDB Cloud address, organization and bucket.
INFLUXDB_MEASUREMENT / INFLUXDB_LOCATION / INFLUXDB_SENSOR	Measurement name, and the location and sensor_type tags on every reading.

Variable	Purpose
TK_CANVAS_WIDTH / TK_CANVAS_HEIGHT / TK_FULLSCREEN	Size of the local Tk display, or full screen.
TIME_ZONE	Station's local time zone.
12H_TIME	Clock times on the display, web pages and alert messages: true (the default) for 2:05 PM, false for 14:05.

Appendix B -- Services and Scheduled Jobs

Service	Purpose
tide	tide.py, the main station process. Unit file: /etc/systemd/system/tide.service.
influxdb3	Local InfluxDB 3 Core database (localhost:8181).
cloudflared	Cloudflare Tunnel connector; all remote access.
apache2	Website and CGI scripts.
grafana-server	Grafana, for viewing the station's data (localhost:3000).
vncserver-x11-serviced or x0vncserver	Remote desktop: RealVNC in front of TigerVNC on new stations, TigerVNC alone on older ones (Section 2.4).
tidemonitor	Headless collector; only on stations that run it instead of tide.

The tide user's crontab runs tideplot.py at 1, 21 and 41 minutes past each hour (`crontab -l -u tide`).

Appendix C -- Files and Locations

Location	Contents
/home/tide/bin/tidegauge/	The repository and per-station files: tide.env, tide_constants.json, sensor_fields.json, keys k1–k3 and ku, .cloud_sync_watermark, and the log newtide.log (with .1 and .2.gz–.7.gz).
/home/tide/.tidenv/	Python virtual environment.
/var/www/html/	Web root: the web pages, tides.db, webimage.png, webinfo.txt, the mail spool (mailspool/ and mailspool/failed/), and the tide.env symlink.
/usr/lib/cgi-bin/	CGI scripts, plus the tidecrypto.py and tideplot.cgi symlinks.
/home/tide/Uploads/tides.db	The periodic database copy (SQL_COPY).
/home/tide/influxdb3/	Local InfluxDB data and plugins.
/etc/systemd/system/	tide.service and the other station unit files.
/var/log/apache2/	Apache access and error logs; the daily visit report reads access.log.1.

Appendix D -- Source Repository

All code referenced in this manual is maintained at github.com/cfi1513840/tide-gauge. The schema reference database (sqltides.db), the configuration templates and the Installation Manual are in the same repository.